Pose Solving Algorithms for Electromagnetic Tracking

Author: Patrick McCarthy
Supervisor: Dr. Andreas Amann

A thesis submitted in partial fulfilment of the
requirements for the degree of
MSc Mathematical Modelling and Machine Learning

School of Mathematical Sciences,
University College Cork,
Ireland

September 2025

Declaration of Authorship

This report is wholly the work of the author, except where explicitly stated otherwise. The source of any material which was not created by the author has been clearly cited.

Date: 26/08/2026

Signature: Patrick McCarthy

Acknowledgements

I would like to thank my supervisor, Andreas Amann, for his guidance and feedback throughout the project, and for his patience in helping me shape both the direction of the work and how it is presented.

I am grateful to Alex Jaeger for his generosity with his time, expertise, and access to hardware throughout this project.

Finally, I would like to thank my family fo rtheir continued support and encouragement, without which this thesis would not have been possible

Abstract

Electromagnetic tracking systems recover sensor pose by solving a nonlinear inverse problem. This report simulates one such Electromagnetic tracking system and characterises three solvers. The standard solver is Levenberg-Marquardt, which is precise but struggles with initialisation on cold start. Neural Networks are less precise but do not have the initialisation issue. This report builds a hybrid solver which uses a neural network to initialise a Levenberg-Marquardt solver, allowing for better precision than neural networks alone, and better reliability than Levenberg-Marquardt alone. On a held-out test set the hybrid solver raises the convergence rate from 0.86 to 0.95 and reduces the 95th percentile positional error from 64.2mm to 1.67mm while running 2.8 times faster than a cold-start Levenberg-Marquardt solver.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Electromagnetic Tracking (EMT) is a tracking technology used to determine the position of a sensor. One of its key benefits is that it does not require line-of-sight, which makes it preferable to methods such as optical tracking, in which the sensor must be in view of a camera.

The Anser EMT [2] developed in UCC in 2017 is one such system. It uses the magnetic field generated by eight coils to estimate the position and orientation (pose) of a small sensor near the field. Recovering a pose from eight coil measurements is a nonlinear inverse problem with no closed form. For that reason, numerical methods are used to solve it. The Levenberg-Marquardt algorithm is the tool used by the Anser system. It is very precise but needs a good initialisation to work. When the system is already running, the previous pose can be used as an initialisation for the next but, when the system is first started, providing a good first initialisation is very important. This report asks the question: how do we construct a good first initialisation?

This report, and its corresponding codebase, provide:

- A Python reimplmentation of the Anser forward model and solver.
- A characterisation of Levenberg-Marquardt's convergence behaviour as a function of initialisation quality.
- A feed-forward neural network trained to predict pose directly.
- A hybrid solver which uses the network output to provide initialisation for Levenebrg-Marquardt.
- A three-way empirical comparison establishing that this hybrid solver is both more reliable and faster.

Chapter 2 introduces the Anser EMT system and explains how it works. We then derive the governing equation, called "The Forward Map", which converts poses into sensor measurements. We then study this map, its properties, and most importantly: when can it be inverted? After this, we take a look at some tools which can be used to solve these types of problems, namely the Levenberg-Marquardt algorithm, and Neural Networks.

Chapter 3 establishes the importance the choice of coordinates and the value of choosing metrics on which to evaluate solvers. It also shows how we implement each of the solvers.

Chapter 4 shows that using Neural Networks to initialise a Levenberg-Marquardt solver both improves reliability and reduces computation time, compared to a cold-start Levenberg-Marquardt solver.

Chapter 2

Theoretical Background

How do we locate the pose of an object in space? One way is using Electromagnetic Tracking (EMT). A magnetic field varies with position in a known way, so a sensor that measures the field carries information about where it is. The signal the sensor measures depends additionally on its orientation relative to the field, so the measurement carries information about orientation too. Current carrying coils create such a field, so a sensor in their presence can measure this generated field and locate itself.

Physics tells us how we can map from pose to measurements, a map we call the *forward map*, but we want to do the opposite. We want to recover the pose given the measurements. Therefore, this is an inverse problem.

The Anser EMT system [2], the first fully open-source EMT platform, solves exactly this problem. It infers the pose (five coordinates) of a small sensor coil from eight measurements induced by a planar array of emitter coils.

This chapter builds the machinery needed to both understand and solve the problem. We first describe the physical system, then derive the forward map from the underlying physics. We then ask when this map can be inverted, at least locally, and turn to methods that can do this in practice.

The first class of methods we look at is the iterative methods, whose success depends on where they start. The second class is neural networks (NNs), which learn an approximate inverse directly. We begin with the physical system itself.

2.1 The Anser EMT System

The Anser EMT system has two main components: a planar field generator, and a tracking sensor coil. This section describes each in turn.

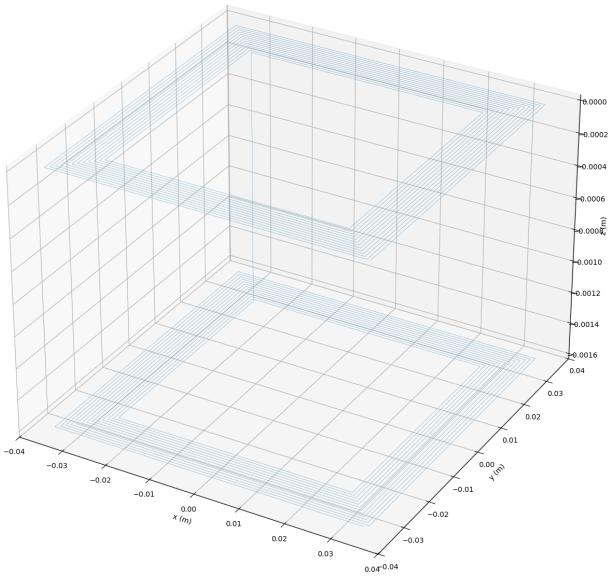

Figure 2.1: Layout of a single emitter coil.

2.1.1 The Field Generator

The Anser EMT system uses a planar field generator consisting of eight being driven at different frequencies. Each coil consists of a wound rectangular spiralling filament which first spirals inward on the top half of a PCB, goes through a via, and the spirals back outwards on the bottom half of the PCB, as seen in Figure 2.1. The two spiral layers together form a single coil. The coil specifications are given in Table 2.1

Eight of these coils are positioned on the PCB, as seen in Figure 2.2.

2.1.2 The Tracking Sensor Coil

The tracking sensor records a measurement related to the flux, and due to the emitter coils running at different frequencies, is able to recover the 8 measurements due to each of the emitter coils after demodulation. These 8 measurements are fed into the pose solving algorithm to determine the position and orientation of the sensor.

Property	Value
Outer side length	70 mm
Turns per coil	25
Track width	0.5 mm
Track spacing	0.25 mm
PCB thickness	1.6 mm

Table 2.1: Specification of PCB emitter coils in Anser field generator [2]

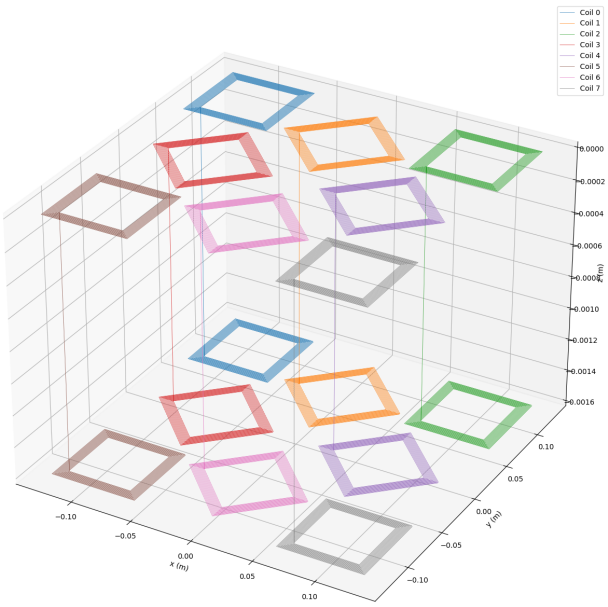

Figure 2.2: Diagram of 8 emitter coils arranged as on PCB

2.2 Magnetic Field Derivation

Each spiral coil is composed of a sequence of straight line filaments and Maxwell's equations in free space are linear [3, p. 59] so the total magnetic field will be the sum of the fields of each individual straight line filament. Our main goal then, is to find the magnetic field generated at an observer point p due to the current flowing along one of these straight line filaments, $\mathbf{a}$.

We start by defining vectors $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{c}$.

- $\mathbf{a}$ points along the current filament.
- $\mathbf{b}$ points from the end of the filament to the observer.
- $\mathbf{c}$ points from the start of the filament to the observer.

Note that throughout we use the convention $|\mathbf{a}| = a$.

Now, we use the Biot Savart Law to find the magnetic field in each of the lines.

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \int \frac{d\mathbf{l} \times \mathbf{r}}{r^3}$$

To evaluate this integral we first parametrise $\mathbf{r}$ in a parameter t, yielding:

$$\begin{aligned} \mathbf{r}(t) &= \mathbf{c} - \mathbf{a}t,\ t \in [0,1] \\ d\mathbf{l} &= \mathbf{a}dt \end{aligned}$$

Substituting in gives:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \int_0^1 \frac{(\mathbf{a}dt) \times (\mathbf{c} - \mathbf{a}t)}{|\mathbf{c} - t\mathbf{a}|^3}$$

Noting that $\mathbf{a} \times \mathbf{a} = 0$, and that $\mathbf{a}$ and $\mathbf{c}$ are constants, this simplifies to:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} (\mathbf{a} \times \mathbf{c}) \int_0^1 \frac{dt}{|\mathbf{c} - t\mathbf{a}|^3}$$

The denominator must now be simplified.

$$|\mathbf{c} - t\mathbf{a}|^3 = \left(c^2 + t^2 a^2 - 2t(\mathbf{a} \cdot \mathbf{c})\right)^{3/2}$$

For simplicity, let $\beta = -(\mathbf{a} \cdot \mathbf{c})$.

$$|\mathbf{c} - t\mathbf{a}|^3 = \left(a^2 t^2 + 2\beta t + c^2\right)^{3/2}$$

Completing the square:

$$|\mathbf{c} - t\mathbf{a}|^3 = a^3 \left(\left(t + \frac{\beta}{a^2} \right)^2 - \frac{\beta^2}{a^4} + \frac{c^2}{a^2} \right)^{3/2}$$

Now let $u = t + \frac{\beta}{a^2}$, $du = dt$, $u(0) = \frac{\beta}{a^2}$, $u(1) = 1 + \frac{\beta}{a^2}$. Also, let $\delta = \frac{c^2}{a^2} - \frac{\beta^2}{a^4}$. Substituting back into our Biot Savart Formula gives:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \frac{\mathbf{a} \times \mathbf{c}}{a^3} \int_{\frac{\beta}{a^2}}^{1+\frac{\beta}{a^2}} \frac{du}{\left(u^2 + \delta\right)^{3/2}}$$

We want to use the identity $\sec^2 \psi = 1 + \tan^2 \psi$, to do so, factor out $\delta^{3/2}$

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \frac{\mathbf{a} \times \mathbf{c}}{a^3 \delta^{3/2}} \int_{\frac{\beta}{a^2}}^{1+\frac{\beta}{a^2}} \frac{du}{\left(\frac{u^2}{\delta} + 1\right)^{3/2}}$$

Now that it is in the correct form let: $\tan^2 \psi = \frac{u^2}{\delta}$, then:

$$\begin{aligned} \tan \psi &= \frac{u}{\sqrt{\delta}} \\ \sec^2 \psi \, d\psi &= \frac{1}{\sqrt{\delta}} du \\ \psi \left(\frac{\beta}{a^2} \right) &= \arctan \left(\frac{\beta}{a^2 \sqrt{\delta}} \right) \\ \psi \left(1 + \frac{\beta}{a^2} \right) &= \arctan \left(\frac{1 + \frac{\beta}{a^2}}{\sqrt{\delta}} \right) \end{aligned}$$

Substituting back in:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \frac{\mathbf{a} \times \mathbf{c}}{a^3 \delta^{3/2}} \int_{\arctan\left(\frac{\beta}{a^2\sqrt{\delta}}\right)}^{\arctan\left(\frac{1}{\sqrt{\delta}}\left(1+\frac{\beta}{a^2}\right)\right)} \frac{\sqrt{\delta} \sec^2 \psi \, d\psi}{\left(1 + \tan^2 \psi\right)^{3/2}}$$

Using the identities $\sec^2 \psi = 1 + \tan^2 \psi$ and $\cos \psi = \frac{1}{\sec \psi}$ and simplifying yields:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \frac{\mathbf{a} \times \mathbf{c}}{a^3 \delta} \int_{\arctan\left(\frac{\beta}{a^2\sqrt{\delta}}\right)}^{\arctan\left(\frac{1}{\sqrt{\delta}}\left(1+\frac{\beta}{a^2}\right)\right)} \cos \psi \, d\psi$$

Solving:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \frac{\mathbf{a} \times \mathbf{c}}{a^3 \delta} \left(\sin\left(\arctan\left(\frac{1}{\sqrt{\delta}} \left(1 + \frac{\beta}{a^2} \right) \right) \right) - \sin\left(\arctan\left(\frac{\beta}{a^2 \sqrt{\delta}} \right) \right) \right)$$

Now, we must consider a right angled triangle to work this out. $\sin\psi = \frac{o}{h}$, $\tan\psi = \frac{o}{a}$ therefore $\sin\left(\arctan\left(\frac{o}{a}\right)\right) = \frac{o}{\sqrt{o^2+a^2}}$ by Pythagoras' Theorem.

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \frac{\mathbf{a} \times \mathbf{c}}{a^3 \delta} \left(\left(\frac{1 + \frac{\beta}{a^2}}{\sqrt{\delta + \left(1 + \frac{\beta}{a^2}\right)^2}} \right) - \left(\frac{\beta}{\sqrt{\beta^2 + a^4 \delta}} \right) \right)$$

Recalling $\delta = \frac{c^2}{a^2} - \frac{\beta^2}{a^4}$, $\beta = -\mathbf{a}\cdot\mathbf{c}$, and looking at each of the denominators:

$$\begin{aligned}
\sqrt{\delta + \left(1 + \frac{\beta}{a^2}\right)^2} &= \sqrt{\frac{c^2}{a^2} - \frac{\beta^2}{a^4} + 1 + 2\frac{\beta}{a^2} + \frac{\beta^2}{a^4}} \\
&= \sqrt{1 + \frac{c^2}{a^2} - \frac{2}{a^2}(\mathbf{a}\cdot\mathbf{c})} \\
&= \frac{1}{a}\sqrt{a^2 + c^2 - 2(\mathbf{a}\cdot\mathbf{c})} \\
&= \frac{1}{a}|\mathbf{a} - \mathbf{c}| \\
&= \frac{b}{a}
\end{aligned}$$

$$\begin{aligned}
\sqrt{\beta^2 + a^4\delta} &= \sqrt{\beta^2 + a^2c^2 - \beta^2} \\
&= \sqrt{a^2c^2} \\
&= ac
\end{aligned}$$

Substituting into our formula for $\mathbf{B}$ now gives:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \frac{\mathbf{a} \times \mathbf{c}}{ac^2 - \frac{(\mathbf{a}\cdot\mathbf{c})^2}{a}} \left(\frac{1 - \frac{\mathbf{a}\cdot\mathbf{c}}{a^2}}{\frac{b}{a}} + \frac{\mathbf{a}\cdot\mathbf{c}}{ac} \right)$$

Multiplying the numerator and denominator across by a

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \frac{\mathbf{a} \times \mathbf{c}}{a^2c^2 - (\mathbf{a}\cdot\mathbf{c})^2} \left(\frac{a^2 - \mathbf{a}\cdot\mathbf{c}}{b} + \frac{\mathbf{a}\cdot\mathbf{c}}{c} \right)$$

Now, picking out these two parts and simplifying, where α is the angle between $\mathbf{a}$ and $\mathbf{c}$:

$$\begin{aligned} a^2c^2 - (\mathbf{a}\cdot\mathbf{c})^2 &= a^2c^2 - a^2c^2\cos^2(\alpha) \\ &= a^2c^2(1-\cos^2\alpha) \\ &= a^2c^2\sin^2\alpha \\ &= |\mathbf{a}\times\mathbf{c}|^2 \end{aligned}$$

$$\begin{aligned} a^2 - \mathbf{a}\cdot\mathbf{c} &= \mathbf{a}\cdot\mathbf{a} - \mathbf{a}\cdot\mathbf{c} \\ &= \mathbf{a}\cdot(\mathbf{a}-\mathbf{c}) \\ &= \mathbf{a}\cdot(-\mathbf{b}) \\ &= -\mathbf{a}\cdot\mathbf{b} \end{aligned}$$

This gives us our final form:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi}\frac{\mathbf{a}\times\mathbf{c}}{|\mathbf{a}\times\mathbf{c}|^2}\left(\frac{\mathbf{a}\cdot\mathbf{c}}{c} - \frac{\mathbf{a}\cdot\mathbf{b}}{b}\right) \tag{2.1}$$

Now, remembering that this equation is linear and that each coil is made up of straight line filaments we get:

$$\mathbf{B}_{\text{total}} = \sum_{\text{coils}}\sum_{\text{filaments}} \mathbf{B}_{\text{filament}}$$

Now, since we are in free space we use the identity $\mathbf{H} = \frac{\mathbf{B}}{\mu_0}$ [3, p. 285]

2.3 Understanding the forward map

The problem is framed as an inverse problem. The forward map takes position and orientation data and calculates 8 measurements from each of the coils. Our goal is to approximate an inverse to this map. To do so, we first try to understand the forward map.

2.3.1 Definition and properties

First we define a pose vector $\mathbf{p} \in \mathbb{R}^5$

$$\mathbf{p} = \begin{bmatrix} x \\ y \\ z \\ \theta \\ \phi \end{bmatrix} \tag{2.2}$$

In practice, $\mathbf{p}$ is restricted to a set $\Omega \subset \mathbb{R}^5$

$$\Omega = \left\{ \begin{bmatrix} x \\ y \\ z \\ \theta \\ \phi \end{bmatrix} \in \mathbb{R}^5 : \begin{array}{c} x \in (-0.125 \text{ m}, 0.125 \text{ m}) \\ y \in (-0.125 \text{ m}, 0.125 \text{ m}) \\ z \in (0.0016 \text{ m}, 0.25 \text{ m}) \\ \theta \in (0 \text{ rad}, \pi \text{ rad}) \\ \phi \in [0 \text{ rad}, 2\pi \text{ rad}) \end{array} \right\} \tag{2.3}$$

Then we define a vector of measurements, $\mathbf{m} \in \mathbb{R}^8$

$$\mathbf{m} = \begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ m_5 \\ m_6 \\ m_7 \\ m_8 \end{bmatrix} \tag{2.4}$$

Now we can write the forward map: $\mathbf{F}(\mathbf{p}) : \Omega \subset \mathbb{R}^5 \to \mathbb{R}^8$. More precisely,

$$m_i = \mathbf{H}_i(x, y, z) \cdot \hat{n}(\theta, \phi) \tag{2.5}$$

where $\mathbf{H}_i$ is the magnetic field calculated at (x, y, z) due to coil i using equation (2.1), and $\hat{n}(\theta, \phi)$ is the vector normal to the sensor, defined by:

$$\hat{n}(\theta, \phi) = \begin{bmatrix} \sin\theta\cos\phi \\ \sin\theta\sin\phi \\ \cos\theta \end{bmatrix} \tag{2.6}$$

Since F is smooth, the image of the 5-dimensional domain, Ω, has measure zero in $\mathbb{R}^8$, therefore F cannot be surjective.

2.3.2 The Jacobian

Now that we have established that F is not surjective we must move our attention to thinking about local invertibility rather than global. To do so we make a local approximation:

$$F(\mathbf{p} + \delta) = F(\mathbf{p}) + J(\mathbf{p})\delta \tag{2.7}$$

where ***delta*** is a small step in $\mathbb{R}^5$, and J is the *Jacobian*, defined by: Jacobian:

Definition 2.3.1 (Jacobian)

$$J(\mathbf{p}) = \frac{\partial F}{\partial \mathbf{p}} \in \mathbb{R}^{8\times 5},\ J_{i,j} = \frac{\partial F_i}{\partial p_j}$$

$\mathbf{H}$ is a smooth map away from the current filaments. $\hat{n}$ is also a smooth map, therefore F will also be smooth away from the plane of the field generator and the Jacobian will be well defined there.

A closed form for $\mathbf{J}$ is cumbersome to find so we instead use finite differences to approximate $\mathbf{J}$

$$\mathbf{J}_{:,\mathbf{i}} \approx \frac{F\left(\mathbf{p} + h\hat{e}_i\right) - F(\mathbf{p})}{h} \tag{2.8}$$

where $\hat{e}_i$ is the unit vector in the ith direction, and h is a small real value.

2.3.3 Local Invertibility

The forward map sends $\Omega \subset \mathbb{R}^5$ into $\mathbb{R}^8$, so it is never globally invertible. This raises the question: when, if ever, is the forward map locally invertible?

The derivative of F at a point $\mathbf{p}_0 \in \Omega$ is the linear map $J_0 = J(\mathbf{p}_0) \in \mathbb{R}^{8\times 5}$. The following conditions are equivalent:

$J_0^\top J_0$ invertible $\iff$ J_0 has full column rank $\iff$ J_0 is injective as a linear map.

It can be shown [4] that if J_0 has full column rank, then F is injective on a neighbourhood of $\mathbf{p}_0$. This means that the pose is unique in that neighbourhood: distinct nearby poses produce distinct measurements, and the pose is locally recoverable from the measurement. As we will see later, in Section 2.4.1, $J^\top J$ is exactly the matrix the Gauss-Newton algorithm must invert.

Rank deficiency therefore signals regions where the pose cannot be recovered from the measurement, even locally. Full column rank is a condition for the inverse problem to be well posed.

2.4 Iterative Methods

Given a function $f : \mathbb{R}^n \to \mathbb{R}^m$ with $m \geq n$, and a given $y \in \mathbb{R}^m$ suppose we want to find an $x \in \mathbb{R}^n$ that satisfies $y = f(x)$. In the case $m = n$ we could get bijectivity which would allow us to simply invert the map, however, in our case we have $n = 5$ and $m = 8$ so we cannot. This leads us to iterative methods.

2.4.1 Gauss-Newton

The Gauss-Newton Algorithm [5] solves this exact kind of problem.

We have a function $f : \mathbb{R}^n \to \mathbb{R}^m$ and we know $y \in \mathbb{R}^m$, we want to find $x_{\text{true}} \in \mathbb{R}^n$ such that $y = f(x_{\text{true}})$. We start with an initial guess $x_0 \in \mathbb{R}^n$ and define a residual:

$$r(x) = f(x) - y \tag{2.9}$$

Then we define a cost function:

$$C(x) = \frac{1}{2} \|r(x)\|^2 = \frac{1}{2} r(x)^\top r(x) \tag{2.10}$$

Suppose now we take a step, $\delta \in \mathbb{R}^n$, we want to find the δ which minimises the cost function.

$$C(x + \delta) = \frac{1}{2} \|r(x + \delta)\|^2 \tag{2.11}$$

Now, linearising:

$$C(x + \delta) \approx \frac{1}{2} \|r(x) + J(x)\delta\|^2 \tag{2.12}$$

Since this is a scalar quantity we can take $\nabla_\delta C$ and set it to zero to find the minimum.

$$\nabla_\delta C(x + \delta) = \left(\frac{\partial}{\partial \delta} C(x + \delta) \right)^\top = 0 \tag{2.13}$$

Now, we consider the kth component of this gradient:

$$(\nabla_\delta C(x + \delta))_k \approx \frac{1}{2} \frac{\partial}{\partial \delta_k} \sum_{j=1}^{m} \left(r_j(x) + \sum_{i=1}^{n} J_{ji}(x)\delta_i \right)^2 \tag{2.14}$$

$$\approx \frac{1}{2} \sum_{j=1}^{m} 2 \left(r_j(x) + \sum_{i=1}^{n} J_{ji}(x)\delta_i \right) \frac{\partial}{\partial \delta_k} \left(r_j(x) + \sum_{i=1}^{n} J_{ji}(x)\delta_i \right) \tag{2.15}$$

$$\approx \sum_{j=1}^{m} J_{jk}(x) \left(r_j(x) + \sum_{i=1}^{n} J_{ji}(x)\delta_i \right) \tag{2.16}$$

$$\approx \sum_{j=1}^{m} J_{jk} r_j + \sum_{j=1}^{m} \sum_{i=1}^{n} J_{jk} J_{ji} \delta_i \tag{2.17}$$

$$\approx \left(J^\top r \right)_k + (J^\top J \delta)_k \tag{2.18}$$

Since this applies for all k we get:

$$\nabla_\delta C(x + \delta) \approx J^\top r + J^\top J \delta \tag{2.19}$$

Setting this equal to zero and solving, yields:

$$\delta = -\left(J^\top J\right)^{-1} J^\top r \tag{2.20}$$

Now that we know the theoretically "best" step to take, we iterate and do this repeatedly:

$$x_{i+1} = x_i - \left(J(x_i)^\top J(x_i)\right)^{-1} J(x_i)^\top r(x_i) \tag{2.21}$$

This method has a known failure mode: it has no step-size control. Far from the solution, where the linearisation is poor, the full step can increase the cost and the iteration can diverge.

2.4.2 Gradient Methods

Gauss-Newton fails when it "trusts" the linear model too far from the point at which it was linearised. The opposite approach makes no model of the residual at all. It just moves in the direction that decreases the cost the fastest. Consider the cost function (2.10) and take its gradient.

$$\nabla_x C(x) = \nabla_x \frac{1}{2} r(x)^\top r(x) \tag{2.22}$$

Using the same argument as before, taking the kth component of this gradient:

$$\begin{aligned}
\left(\nabla_x C(x)\right)_k &= \frac{1}{2}\frac{\partial}{\partial x_k}\sum_{j=1}^{m} r_j(x) r_j(x) && (2.23)\\
&= \frac{1}{2}\sum_{j=1}^{m}\frac{\partial}{\partial x_k} r_j(x)^2 && (2.24)\\
&= \frac{1}{2}\sum_{j=1}^{m} 2 r_j(x)\frac{\partial r_j(x)}{\partial x_k} && (2.25)\\
&= \sum_{j=1}^{m} r_j(x)\frac{\partial (f_j(x) - y_j)}{\partial x_k} && (2.26)\\
&= \sum_{j=1}^{m} r_j(x)\frac{\partial f_j(x)}{\partial x_k} && (2.27)\\
&= \sum_{j=1}^{m} r_j(x) J_{jk}(x) && (2.28)\\
&= \left(J^\top(x) r(x)\right)_k && (2.29)
\end{aligned}$$

Gradient descent then steps against this gradient, with the step-size being denoted by the parameter $\eta \in \mathbb{R}$:

$$x_{i+1} = x_i - \eta J(x_i)^\top r(x_i) \tag{2.30}$$

Because $-\nabla_x C$ is in the direction of steepest local decrease, a sufficiently small η is guaranteed to reduce the cost, so the method is robust far from the solution where Gauss-Newton diverges. Its failure mode is also the opposite of Gauss-Newton's. Near the solution, the gradient becomes small, and since the algorithm has no knowledge of the distance to the solution it bounces back and forth around the minimum.

2.4.3 Levenberg's Contribution

In 1944, Levenberg [6] interpolated between these two algorithms. If we denote each step as δ_{GN} and δ_{GD}, for Gauss-Newton and gradient descent, respectively we get that each of these steps solves the equations:

$$(J^\top J)\delta_{GN} = -J^\top r \tag{2.31}$$

$$\delta_{GD} = -\eta J^\top r \tag{2.32}$$

Now, letting $\lambda = \frac{1}{\eta}$, multiplying by $I \in \mathbb{R}^{n \times n}$, and interpolating between these two yields:

$$\left(J^\top J + \lambda I\right)\delta = -J^\top r \tag{2.33}$$

Here, λ is known as the damping factor, and another key insight Levenberg had was to vary λ. Since GN works better the closer to the solution we get and GD works better the further from the solution we are, the damping is varied dynamically.
We start at step i with $\lambda = \lambda_i$. If $C(x_{i+1}) < C(x_i)$ we are moving in the right direction so can move more into the GN regime, setting $\lambda_{i+1} = 0.1\lambda_i$. However, if $C(x_{i+1}) \geq C(x_i)$ then it is clear that GN is not working and we need to move into a GD dominated regime, thus setting $\lambda_{i+1} = 10\lambda_i$. This ensures that when we are far from the solution we are in a GD dominated regime and as we get closer to the solution GN starts to take over.

2.4.4 The Levenberg Marquardt Algorithm

One downfall of Levenberg's approach is that λ is the same in every dimension, even though the dimensions may operate on different scales. Marquardt [7] solves this by replacing the identity matrix, I with $D = \text{Diag}\left(J^\top J\right)$ to scale the weights, where Diag is the operator which acts on matrices by setting the off-diagonal elements to zero and operates on vectors by making a

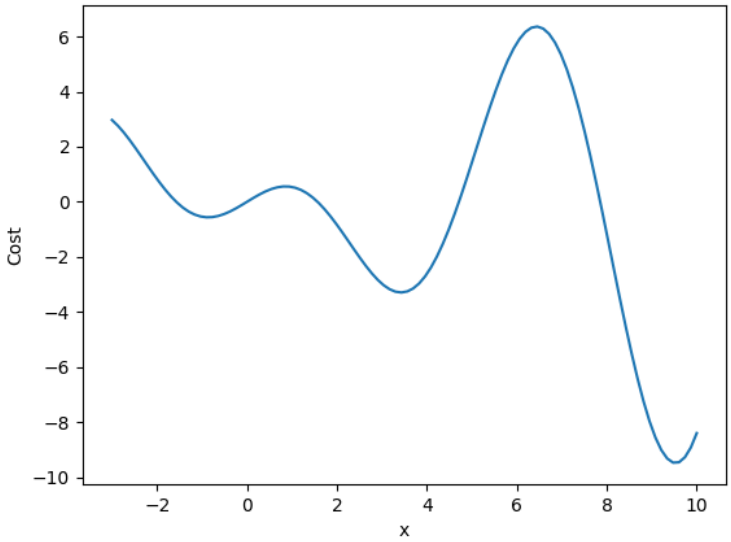

Figure 2.3: Example showing multiple local minima using $y = x \cos x$, $x \in (-3, 10)$

diagonal matrix out of its components. The Levenberg-Marquardt algorithm solves the following equation at every iteration:

$$\left(J^\top J + \lambda D\right) \delta = -J^\top r \tag{2.34}$$

This means that the algorithm works with different damping in each direction, proportional to the "scale" of that particular direction.

2.4.5 Convergence and the Role of Initialisation

Both Gauss-Newton and Levenberg-Marquardt are local methods: they converge to a stationary point of C, and which stationary point depends on where they start. Since C is typically non-convex it can have multiple local minima, an example is shown in Figure 2.3. In our problem, in particular, given the high levels of symmetry between the coils, it is highly likely that there are multiple minima. These iterative methods work when the initial guess is in the correct basin of attraction. That then raises the question, how do we get into the right basin of attraction?

2.5 Neural Networks for learning inverses

The iterative methods above invert the forward map each time for every measurement. An alternative would be to spend the computation time upfront and approximate a full inverse map once.

2.5.1 Feed Forward Neural Network

We use a fully-connected feed forward neural network (FFNN) to approximate the inverse. Suppose we have a function $\Phi : \mathbb{R}^N \to \mathbb{R}^M$ an approximate inverse is defined on the image of Φ, which we call f. We would like to find $y \in \mathbb{R}^N$ such that $y = f(x), x \in \mathbb{R}^M$. To do so, a FFNN with L hidden layers performs the following composition, with $o_0 = x$ and $o_L = \hat{y}$:

$$o_{i+1} = \sigma\left(W_i o_i + b_i\right) \tag{2.35}$$

where W_i and b_i are the weight matrices and bias vectors (collectively the parameters w), and σ is the nonlinear activation function.
We represent this approximation as $\hat{f}_w : \mathbb{R}^M \to \mathbb{R}^N$, with $\hat{y} = \hat{f}_w(x)$

The question then becomes how do we find the collection of parameters, w, which does this?

2.5.2 Training a FFNN

Because we possess F we can generate unlimited training data, a collection of labelled pairs, (x_i, y_i). The goal of training is then to minimise the *loss function* with respect to the parameters, w. The loss function is a performance metric for the network, and we will see in Section 4.3 that the choice of loss function is crucial to make a network perform well. We minimise the loss using an optimiser, in our case the Adam optimiser [8]. Generalisation is then assessed on a held-out test set never seen during training.

Chapter 3

Methods

Levenberg-Marquardt converges to a stationary point of the cost function, and which stationary point it reaches is determined by where it begins. So, how do we get into the right basin?

The solver is only as good as its initial guess. This chapter builds a neural-network based initialiser which then provides an initial guess for LM to allow it to converge to the correct stationary point.

In this chapter, we follow a pipeline. First we figure out how to sample poses uniformly in Ω. Then, these poses are fed into the forward model, yielding eight measuremnts. These 8 measurements are passed to a solver which gives back a predicted pose. The pipeline is shown in Figure 3.1. The first two stages of the pipeline exist only at training time. At inference time, the system only executes the three remaining stages.

Section 3.1 implements the forward map from Section 2.3 in Python and shows how to sample poses uniformly. Section 3.2 defines the positional and angular errors used to evaluate the solvers. Section 3.3 describes the Levenberg-Marquardt solver, and shows implementation considerations. Section 3.4 examines how the pose should be represented as a target, particularly with respect to angle. Section 3.5 describes the architecture and training of the neural-network solver.. Section 3.6 joins the two halves.

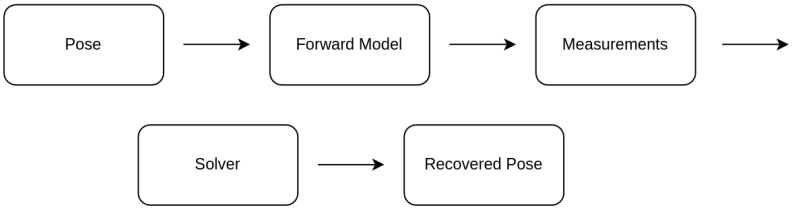

Figure 3.1: Pose estimation pipeline.

3.1 Data generation

3.1.1 Anser Simulation in Python

We simulate the Anser system in Python, developing on a MATLAB simulation by Jaeger et al. [2]. [1] The forward model proceeds as follows:

Coil geometry First we generate the vertex coordinates for a square spiral PCB emitter coil. This is done in a local coordinate system and then copies are rotated and translated to place them in the global coordinate system.

Each coil is a square which spirals in, it has N turns per layer, the side-length of the outermost square is l, the trace width is w, there is a line to line spacing of s and the thickness of the PCB is z_{thick}. After the coil has done N_{turns} turns on the upper layer, it is connected with a via to the lower layer where the coil spirals out again.

Conversion from Local to Global Coordinates As all of the coils are the same shape, only one coil is generated, then it is copied a number of times into different places in space. The module selects an axis about which the rotation takes place. This is represented as an integer $(0, 1, 2) - (x, y, z)$. This works nicely because the rotation matrix can be generated using an even permutation of the axes. The module then applies the permuted rotation matrix to a copy of the input vector and then applies a translation.

Field Calculation The calculation of the magnetic field generated by the coils is done in `field_coil_calc`. It uses the linearity of the Biot-Savart line integral to calculate the total magnetic field due to the coils at an observer

[1] The implementation is available at `https://github.com/pmccarthy3901/ansermodelling`. A mapping from the sections of this chapter to the corresponding modules is given in Appendix .1.

Variable	Min (mm)	Max (mm)
x	-125	125
y	-125	125
z	1.6	250

Table 3.1: Bounds of the sampling volume.

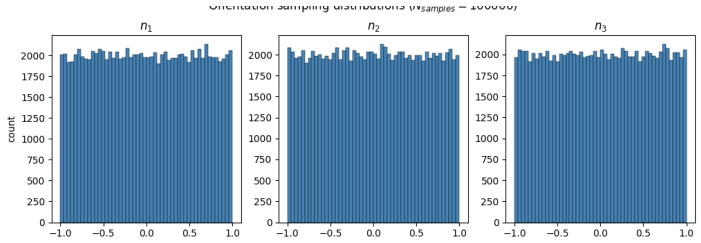

Figure 3.2: Orientation normal vectors sampled.

point as the sum of the contributions of each straight-line segment. It then implements equation (2.1) to calculate the magnetic field.

3.1.2 Sampling

When generating data, it is important that our sampling space is uniform. For position, this is easy, we sample uniformly in the bounds of the box. The angle, however requires a bit more consideration.

3.1.2.1 Sampling the position

The Anser system operates in a 25x25x25cm box.

Since the sensor coil is on a PCB that is 1.6mm thick, the z value has a minimum of $z = 0.0016$m. Therefore we sample uniformly within the bounds shown in Table 3.1

3.1.2.2 Sampling the orientation

Consider an area element dA on the unit sphere, seen in Figure 3.3. Then the area element is $dA = \sin\theta d\theta d\phi$. Therefore, to uniformly sample on the sphere we must not sample uniformly in θ, but rather in $\cos\theta$. The orientation sampling is shown in Table 3.2. As seen in Figure 3.2, this leads to uniform sampling on the sphere.

Variable	Min	Max
$\cos\theta$	-1	1
ϕ	0 rad	2π rad

Table 3.2: Bounds of the orientation sampling volume.

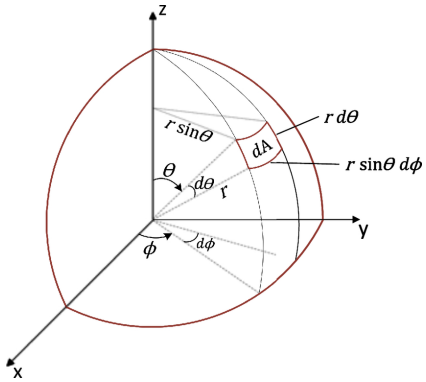

Figure 3.3: Area element on the unit sphere. Reproduced from [1]

3.1.3 Dataset size, split, and normalisation

The dataset consists of pose-measurement pairs. A subset of the poses are shown in space in Figure 3.4. The measurements are the model input and the pose is the regression target. The data is split 80/20 into training and validation sets, and the validation set is never seen during training. Models are then evaluated on a held out test set. The distribution of datapoints is shown in Table 3.3

3.2 Error metrics

In order to quantify how "good" a particular model is, we need to define our errors. Since the pose has two components: position and orientation, we

Split	Size
N_{train}	$80,000$
N_{val}	$20,000$
N_{test}	$20,000$

Table 3.3: Data splits between training, validation, and testing data.

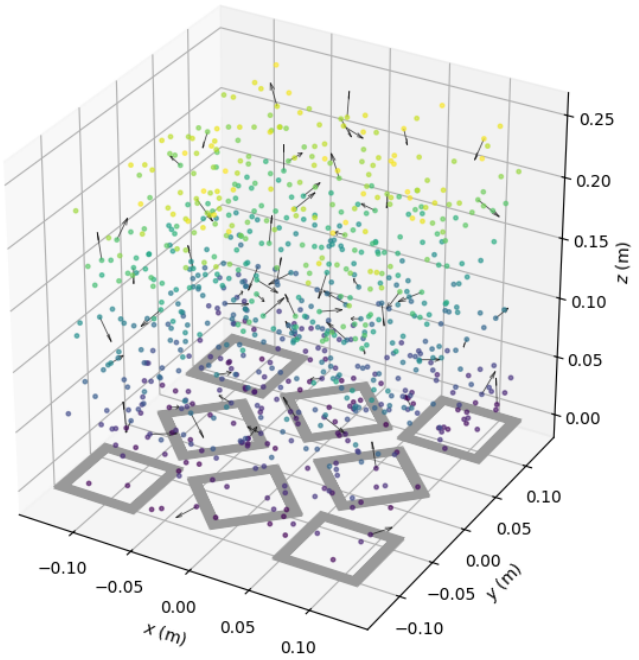

Figure 3.4: 800 poses sampled with some arrows indicating orientation.

define an error on each.

3.2.1 Positional Error

The positional error e_p is defined using the euclidean distance between the true position, x_{true}, and the predicted position, x_{pred}.

$$e_p := \|x_{\text{true}} - x_{\text{pred}}\|_2 \tag{3.1}$$

3.2.2 Angular Error

The angular error, $e_{\hat{n}}$, is defined in relation to the unit normal orientations, where $\hat{n}_{\text{true}}$ is the true unit normal, and $\hat{n}_{\text{pred}}$ is the predicted unit normal:

$$e_{\hat{n}} := \arccos\left(\hat{n}_{\text{true}} \cdot \hat{n}_{\text{pred}}\right) \tag{3.2}$$

3.3 The Levenberg-Marquardt solver

The previous chapter derived the Levenberg-Marquardt update function (2.34). This section describes the implementation and parameter choice. The chosen

Property	Value
λ_0 (Initial damping)	10^{-5}
$\lambda_{\max}$ (Maximum damping)	10^7
ϵ_{grad} (Gradient threshold)	10^{-5}
Maximum Iterations	100
h (Jacobian finite difference)	10^{-5}

Table 3.4: Parameters used in LM implementation.

parameters were tuned against the validation set and are shown in Table 3.4

3.3.1 Implementation

The solver is implemented in NumPy (`models/LM_solve.py`). Its components are as follows:

Jacobian. As discussed in the previous chapter, we approximate the Jacobian using finite difference, as in (2.8).

Scaling. Following Marquardt's implementation [7], the damping matrix is given by:

$$D = \text{diag}(\max(\text{diag}(J^\top J), 10^{-12})) \tag{3.3}$$

where the floor of 10^{-12} guards against a zero diagonal entry making D singular and diag is the function defined in Section 2.4.3

Constraint Handling. At each iteration, constraints are applied to both the position and orientation. The position is clipped to within the box shown in Table 3.1. The orientations are handled by folding θ through the pole with a π shift in ϕ, and wrapping ϕ. This keeps the orientation unchanged on the unit sphere and alters only its coordinates.

Damping. Damping is done as in Section 2.4.3.

Stopping. Iteration terminates when any of three conditions are met:

1. the l_∞ norm of the gradient falls below a threshold, ϵ_{grad}.
2. The damping parameter, λ exceeds $\lambda_{\max}$, indicating that no downhill step can be found.

3. A maximum number of iterations is reached, indicating that the solver cannot find a solution.

Only the first of those three stopping criteria indicates success, and even then, the solver successfully reaching a minimum does not mean that it has successfully reached the correct minimum. Therefore we have to define convergence.

We say that a solution has converged if for some ϵ_p and some ϵ_n "acceptable errors":

$$e_p < \epsilon_p$$
$$e_{\hat{n}} < \epsilon_n$$

3.4 Representing the pose

The pose contains two angles, and how they are represented as regression targets matters more than it first appears. Regressing against θ and ϕ directly has three problems. First, ϕ is periodic: $\phi = 0.01$ and $\phi = 2\pi - 0.01$ are only separated by 0.02 rad, however a naive loss would heavily punish such a situation. The second problem is with ϕ: at the poles $\theta \in \{0, \pi\}$, ϕ is not defined. The third problem is more general: squared error in the angles is not a distance on the sphere. The metric induced on the unit sphere, S^2, by the parametrisation carries an extra factor of $\sin^2\theta$:

$$ds^2 = d\theta^2 + \sin^2\theta \, d\phi^2 \tag{3.4}$$

For these reasons, it becomes natural to represent the orientation not in terms of angles, but in terms of the unit normal itself:

$$\hat{n}(\theta, \phi) = \begin{bmatrix} \sin\theta\cos\phi \\ \sin\theta\sin\phi \\ \cos\theta \end{bmatrix} \tag{3.5}$$

The normal vector is smooth, unique, and has no wrap-around. We therefore train networks whose target is the vector:

$$t = (x, y, z, n_1, n_2, n_3) \tag{3.6}$$

where n_i is the unit normal in the ith direction.

This argument, however, does not apply to the Levenberg-Marquardt solver of Section 3.3, which continues to operate in the (θ, ϕ) regime. This is because an iterative solver initialised near its solution does not encounter

Property	Value
Input dimension	8 (measurements)
Output dimension	6 (x, y, z, n_1, n_2, n_3)
Hidden layers	3
Units per layer	(256,256,256)
Activation	tanh

Table 3.5: Network architecture.

these problems and, conversely, if we were to regress against the unit normal we would have the issue of having 6 parameters for 5 degrees of freedom meaning that $J^\top J$ would not be invertible, contradicting the exact condition that LM needs.

3.5 Neural Network Solver

Section 3.3 establishes that the LM solver needs a good initial guess to converge. This section builds a neural network solver, the main advantage of which is its ability to learn the global configuration of the system and thus work on a "cold start". This section shows the choice of architecture, loss function, and training methods.

3.5.1 Neural Network Architecture

The neural network solver uses a basic Feed Forward Neural Network (FFNN) which takes the 8 input measurements and outputs the 6 dimensional pose (x, y, z, n_1, n_2, n_3). The neural network architecture is shown in Table 3.5

3.5.2 Choice of loss function

In Section 3.2 we defined two errors: e_p, the positional error, and $e_{\hat{n}}$, the angular error. Now we must define a loss function. The only problem is that these errors operate on completely different scales. Given the bounds of our box, e_p varies between 0 and w_p, where w_p is the diagonal length of the box, and $e_{\hat{n}}$ varies between 0 and $w_{\hat{n}} = \pi$. To give equal weight to both components we define our loss, $\mathcal{L}$, as the weighted sum between squares of the two as follows:

$$\mathcal{L} = e_p^2 + \frac{w_p^2}{w_{\hat{n}}^2} e_{\hat{n}}^2 \tag{3.7}$$

Parameter	Value
Optimiser	Adam
Schedule	Cosine Annealing
Batch size	32
Epochs	200
η_0	10^{-3}
$\eta_{\min}$	0
$T_{\max}$	200

Table 3.6: Training configuration.

This weight is chosen such that the two terms have equal range over the sampling volume.

3.5.3 Neural Network Training

The network is trained with the Adam optimiser [8] to minimise (3.7) over the dataset described in Section 3.1, using the settings and hyperparameters described in Table 3.6

The learning rate uses a cosine scheduler [9], decaying from its initial value to zero, as in equation (3.8) with parameters in Table 3.6, where t is the current epoch and η_0 is the learning rate at the 0th epoch. This allows for a high learning rate at the start, which then decays smoothly at every epoch. This means that at the start the model learns quickly, and at the end the model has sufficiently small learning rate to not oscillate around the minimum.

$$\eta_{t+1} = \eta_{\min} + (\eta_t - \eta_{\min}) \frac{1 + \cos\left(\frac{t+1}{T_{\max}}\pi\right)}{1 + \cos\left(\frac{t}{T_{\max}}\pi\right)} \tag{3.8}$$

3.6 Hybrid Model

Sections 3.3 and 3.5 present a LM based solver and a NN solver, respectively. The shortcomings of these two solvers are complementary. A hybrid model combines these two, feeding the output of the NN model into a LM solver. This allows the NN to provide a good initialisation for the LM solver to further refine.

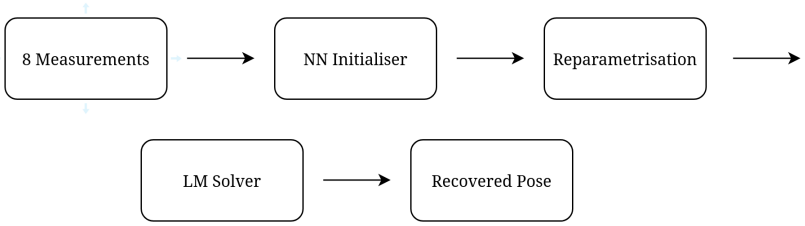

Figure 3.5: Data pipeline for hybrid model

3.6.1 Coupling the two solvers

The two components operate in different parametrisations. The NN uses (x, y, z, n_1, n_2, n_3) and the LM solver uses (x, y, z, θ, ϕ). This means that before feeding the NN output into the LM solver, we have to convert the parametrisation.

Given the predicted normal $\hat{n} = (n_1, n_2, n_3)$, normalised to unit length, the corresponding θ and ϕ are given by:

$$\begin{aligned} \theta &= \arccos(n_3) \\ \phi &= \text{atan2}(n_2, n_1) \end{aligned}$$

where atan2 is the two argument arctangent defined by:

$$\text{atan2}(y, x) = \begin{cases} \arctan\left(\frac{y}{x}\right) & \text{if } x > 0, \\ \arctan\left(\frac{y}{x}\right) + \pi & \text{if } x < 0 \text{ and } y \geq 0, \\ \arctan\left(\frac{y}{x}\right) - \pi & \text{if } x < 0 \text{ and } y < 0, \\ +\frac{\pi}{2} & \text{if } x = 0 \text{ and } y > 0, \\ -\frac{\pi}{2} & \text{if } x = 0 \text{ and } y < 0, \\ \text{undefined} & \text{if } x = 0 \text{ and } y = 0. \end{cases} \tag{3.9}$$

The use of atan2 instead of the normal arctangent allows us to both get in the correct quadrant and also mitigates against the singularity caused when $n_1 = 0$.

Now we can build our hybrid pipeline. The pipeline is shown in Figure 3.5

Chapter 4

Results

Chapter 3 builds three solvers. This chapter evaluates each of the three solvers and compares them against one another.

4.1 Experimental Setup

All models are evaluated on the same held out test set consisting of N_{test} datapoints, generated seperately from the training/validation datapoints. The data splits are shown in Table 3.3.

4.1.1 Hardware Setup

Training and inference times are reported throughout this chapter. All computation is done on the same machine with specifications shown in Table 4.1. Models are trained purely on the CPU. In reporting we use the CPU time, not the wall time, because some processes are singlethreaded whereas others are multithreaded. For example, NN training is multi-threaded, whereas LM inference is single-threaded.

Component	Model
CPU	AMD Ryzen 5 1600x
RAM	32GB DDR4 2133MT/S

Table 4.1: Computer specifications for training and inference

Parameter	Value
ϵ_p	1 mm
ϵ_n	1°

Table 4.2: Convergence criteria for results reporting.

4.1.2 Convergence Criteria

In Section 3.3 we define two constants, ϵ_p and ϵ_n, which dictate when we consider a run to have *converged*. In this chapter, we use the values shown in Table 4.2. A solve is said to have converged if the positional and angular errors are both less than ϵ_p and ϵ_n, respectively.

4.2 The Levenberg-Marquardt Solver

Section 3.3 shows how we implement the Levenberg-Marquardt solver. This section shows its performance and motivates the need for Neural Network Initialisation.

4.2.1 Basin Sweep

We want to know how accurate an initialisation must be before LM can recover the pose. We therefore work out how "bad" an initialisation can be to give a desired convergence rate.

Method. $N_{\text{bs}} = 200$ poses are sampled from the held-out test set with a fixed seed. We then perform two independent perturbation sweeps: A positional sweep and an angular sweep. The position perturbation is generated by picking a random direction via Gaussian sampling. Then normalising that vector and multiplying by the perturbation distance, d. The angular perturbation requires some consideration.

If we want to perturb an orientation vector $\hat{n} = (n_1, n_2, n_3)$ by an angle α, we generate a random vector from a Gaussian distribution $v \sim \mathcal{N}(0, I_3)$. Then we project v onto the plane perpendicular to $\hat{n}$, as in equation (4.1). After this, we normalise $v_\perp$ and rotate by α in the direction of $v_\perp$ as in equation (4.2).

$$v_\perp = v - (v \cdot \hat{n})\hat{n} \tag{4.1}$$

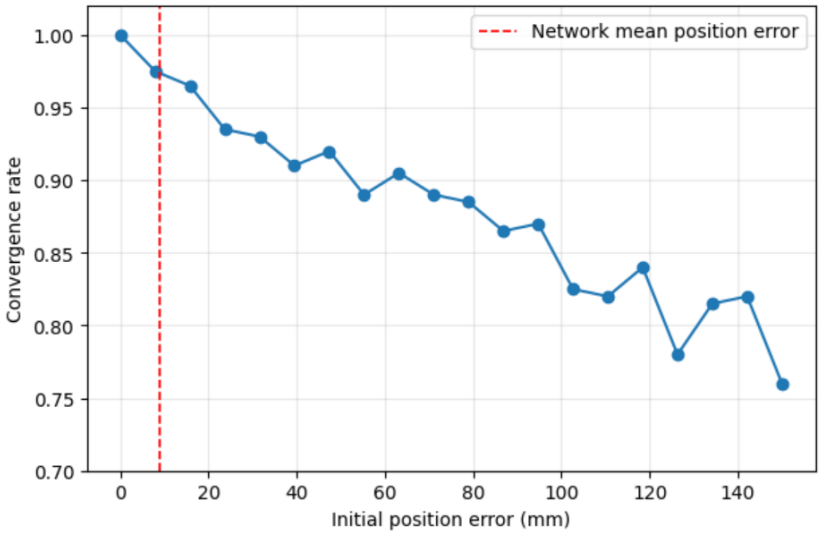

Figure 4.1: Convergence rate against positional perturbation. The dashed line marks the mean positional error of the trained Neural Network seen in Section 4.3.

$$\hat{n}_{\text{pert}} = \cos(\alpha)\hat{n} + \sin(\alpha)(v_{\perp}/\left|v_{\perp}\right|) \tag{4.2}$$

For each of the sweeps, the convergence rate is calculated as in Section 4.1.2.

Positional Sweep. The position is swept from $d = 0$ to $d = 150$mm. Figure 4.1 shows that as the perturbation magnitude increases, there is a steady decrease in the convergence rate, with no sharp dropoff. This suggests that there are no sharp boundaries contrary to what one might expect of a Newton-type method.

Angular Sweep. The angle is swept from $\alpha = 0$ to $\alpha = 90°$. Figure 4.2 shows the same story as the positional sweep, steady decrease with no sharp dropoff. We do note that the angle is a lot more forgiving than position, retaining a 97.5% convergence rate up to $\alpha = 42.6°$. This is because the equation (2.5) depends on $1/r^3$ whereas the orientation angle only enters through the dot product.

What is interesting in both the positional and angular sweeps is that the failure is discrete. Either the solver converges to machine precision $\sim 10^{-14}$mm/° or else it is off by a large number. This is seen when comparing the mean, median, and 95% error. For example, in the positional

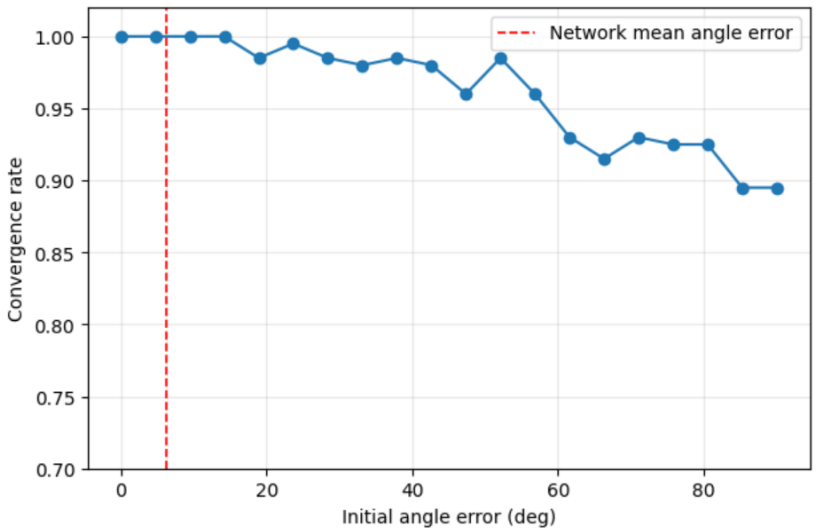

Figure 4.2: Convergence rate against angular perturbation. The dashed line marks the mean angular error of the trained Neural Network.

perturbation with $d = 86.8$ mm we get a mean positional error of 8.13mm but a median positional error of $1.8 * 10^{-11}$mm, suggesting that a few outliers inflate the mean. This is further confirmed by the 95% error of 56.3mm, showing that either the solver works or it does not, there is no slow dropoff in error. This is also shown in Figure 4.3

4.3 Network Training

4.3.1 Training Configuration

In training the network, a number of tuning choices were made against the validation set. First, and most importantly, the network uses the 6-dimensional (x, y, z, n_1, n_2, n_3) pose representation rather than the 5-dimensional (x, y, z, θ, ϕ) representation. Secondly the "PoseLoss" loss function described in Section 3.5.2 was used, instead of MSE. Third, the learning rate was varied using a cosine annealing scheduler, as described in section 3.4. Each of these training configurations are shown in Table 4.3

Base. Table 4.4 shows that the base model performs only marginally better than a baseline estimator i.e. it is not much better than a random guess. The expected positional and angular error for two randomly sampled points were estimated using a Monte Carlo simulation drawing from the test set. More

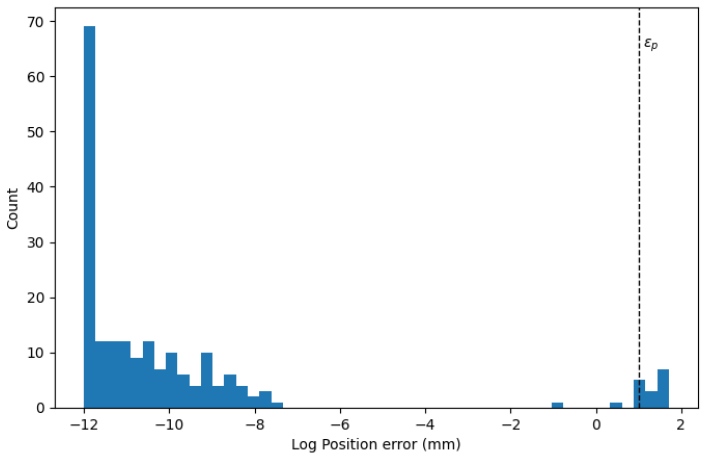

Figure 4.3: Histogram of log positional error for $d = 47.4$mm showing two error classes, failure and convergence.

Name	Target	Loss	LR Scheduler	Optimiser	Epochs
Base	(θ, ϕ)	MSE	None	Adam	200
Normal	$\hat{n}$	MSE	None	Adam	200
PoseLoss	$\hat{n}$	PoseLoss	None	Adam	200
Final	$\hat{n}$	PoseLoss	Cosine	Adam	200

Table 4.3: Experimental Design of NN models.

Name	Mean Pos Error (mm)	Mean Angular Error (deg)
Random Pair	165	90
Centre Predictor	120	90
Base	135	87.1
Normal	139	47.7
PoseLoss	21.3	8.67
Final	9.2	6.43

Table 4.4: Test set performance of NN models.

interestingly, the base model actually performs worse on position than just guessing the centre of the box every time. This tells us that the base model learns nothing about the orientation and almost nothing about the position.

Normal. The "Normal" configuration improves the angular error to about half of baseline but left the positional error the same, suggesting that the change to normal parametrisation allows the network to start learning the orientation configuration of the system.

PoseLoss. The biggest change happens when we change the loss function from MSE to our PoseLoss, reducing positional error by a factor of 6.5 and angular error by a factor of 5.5. This change allows the network to actually learn against angular error and also weights the positional and angular error more equally.

Final. Further refinement is achieved through the use of Cosine learning rate scheduling, allowing training to slow down towards the end, learning the finer details of the data.

4.3.2 Final Model

The Final NN model, with configuration seen in Table 4.3 was chosen to be used. The number of epochs was chosen to be 200 because by 200 the error on the validation set started plateauing. This plateau can be seen in Figure 4.4.

4.4 Three-way Comparison

All three solvers are evaluated on the same held-out test set of 20,000 datapoints. The LM solver uses the same parameters as in section 3.3. The LM model is initialised from the centre of the box with the orientation in the $\hat{z}$ direction. The hybrid solver uses the same solver parameters and differs only in where it starts. The hybrid solver uses the output of the NN as a start point. The results of this test are shown in Table 4.5.

The LM and NN solvers obtain similar mean positional errors, however these solvers operate very differently. The LM solver is extremely bimodal, it either converges to a tiny error on the order of 10^{-11}mm or else it completely fails. This behaviour is seen when we look at the median and 95th percentile position errors. The NN, on the other hand, has a unimodal distribution of positional errors, with mean, median, and 95th percentile error on similar

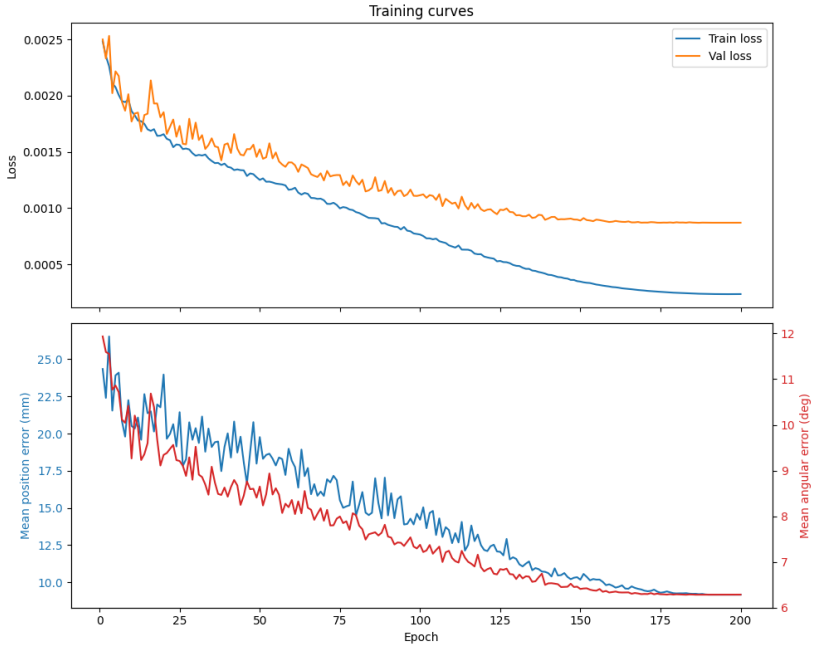

Figure 4.4: Loss curves and error curves on the validation set

	LM (cold start)	Network	Hybrid
Mean position error (mm)	9.71	9.20	1.12
Median position error (mm)	1.5×10^{-11}	7.33	3.8×10^{-12}
95th pct. position error (mm)	64.2	21.7	1.67
Mean angular error (°)	8.95	6.43	1.89
Median angular error (°)	5.3×10^{-12}	3.62	1.2×10^{-12}
95th pct. angular error (°)	79.5	20.9	2.46
Convergence rate	0.859	0.0008	0.947
Training time	—	2h 32m 49s	—
Inference time	16m 3s	361 ms	5m 45s

Table 4.5: Performance of the three solvers on the held-out test set of 20,000 poses.

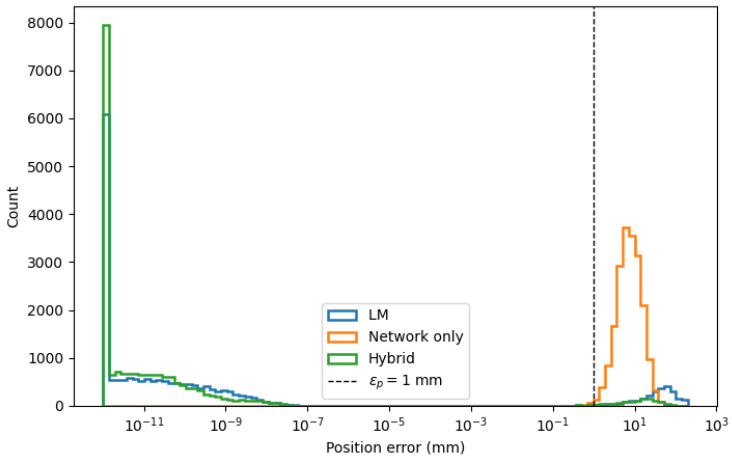

Figure 4.5: Histogram of positional errors for each of the three solvers.

scales. This suggests that NN performs similarly on almost all poses. This behaviour is seen in Figures 4.5 and 4.6

The goal of the Hybrid solver is to push the LM into the low error mode. We see in Table 4.5 that it succeeds in this goal. It decreases the rate of non-convergence by a factor of 2.66. The median errors of the Hybrid solver remain similar to that of the LM solver, however the mean and 95th percentile errors decrease drastically. Looking at the rightmost peaks in Figures 4.5 and 4.6 shows us that the Hybrid has fewer failures and even when it does fail it does not fail the error is smaller than the LM solver. The Network initialisation does not make LM more accurate, it just makes LM succeed more often.

The LM solver doesn't require any training, all of the compute is done at inference time. The NN solver on the other hand, does a vast majority of the compute at training time, and only a small amount at inference time. Table 4.5 shows that them LM solver requires two orders of magnitude more inference time than the NN, the NN's cost is instead paid at training time, taking over two hours to train. The Hybrid solver inherits the compute done in NN training, making inference faster by making it do fewer iterations. On the test-set it takes the Hybrid solver just over five minutes to solve all N_{test} poses. This is three times faster than the LM solver. This means that after approximately 300,000 poses the Hybrid solver will reach its break-even point. Further advocating for the Hybrid, inference time should be valued higher than training time, as training only has to be done once and can be done on a powerful computer rather than on the system itself.

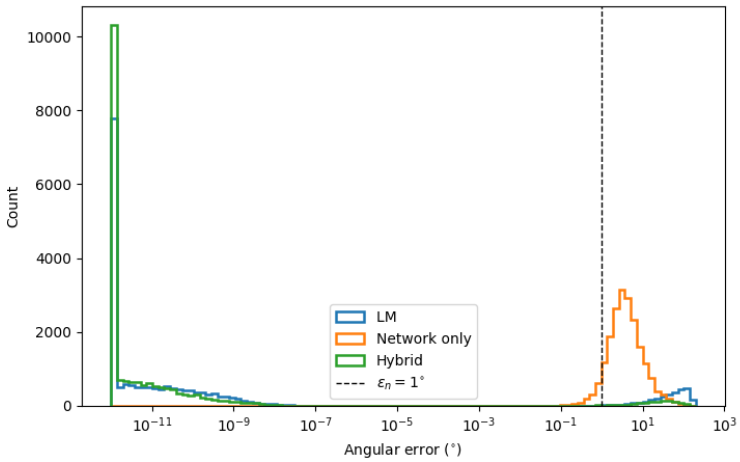

Figure 4.6: Histogram of angular errors for each of the three solvers.

4.5 Discussion of Results

Does the basin sweep explain the hybrid? The basin sweep predicts a convergence rate of 0.975 at the network's position error and a convergence rate of 1.00 at the NN's angular error. However, the Hybrid obtains a convergence rate of 0.947, where one might expect a convergence rate of $(1.00 * 0.975) = 0.975$. Two possible reasons present themselves. Firstly, the basin sweep is only calculated on 200 points compared to the 20,000 of the hybrid analysis, so the difference in convergence rates might be an artifact of the different sample sizes. The second possible reason is that the composition of errors might not be independent. We test the first by constructing a hypothesis test. We test the null hypothesis that both are estimates of the same underlying rate. We compute the z-score in (4.3) to be 2.51 giving a p value of 0.012. We therefore reject the null hypothesis at 5% significance. This suggests that the composition of errors is therefore not independent. This means that the sweeps explain why the Hybrid performs well, but cannot predict how well the Hybrid performs.

$$z = \frac{0.975 - 0.947}{\sqrt{\frac{0.975 \times 0.025}{200} + \frac{0.947 \times 0.053}{20000}}} = 2.51 \tag{4.3}$$

Consistency between basin sweep and LM solver. The LM solver initialises at the box centre, with mean distance to a test pose of approximately 120mm and gives a convergence rate of 0.859. The basin sweep gives

a convergence rate of 0.84 when the pose is perturbed 118mm from the true pose. Using the same test as in the previous section, we get a p value of 0.64. The two are statistically indistinguishable from one another at this sample size, suggesting that the two experiments are consistent with one another.

Precision vs Reliability. Throughout this chapter, we see two key attributes that solvers have. Precision, and reliability. LM is very precise, when it receives a good initialisation. It's not very reliable though. If it doesn't receive a good initialisation is completely fails. For example, in the basin sweep with $d = 150$mm, the 95th percentile positional error is 150mm, meaning that the solver did nothing to the initial guess. The NN on the other hand isn't very precise, with positional errors consistently on the order of 9mm. However, it is extremely reliable, achieving a positional error less than 21.7mm 95% of the time. The function of the Hybrid solver is that it inherits precision from the LM solver, and reliability from the NN solver. None of the improvements in this chapter came from giving the model more training data or increasing the model's capacity. One came from the choice of loss function, and one came from choosing to use the NN to initialise LM.

Chapter 5

Conclusion

5.1 Limitations and Future Work

Noise. All results are noiseless. Real EMT systems experience two types of noise. Random noise, and systematic noise. Random noise comes from the sensitivity of the sensors and compounding errors in the data pipeline. Systematic noise which comes from nearby conductive or ferromagnetic objects. These two forms of noise together are the dominant error source in real EMT deployments.

Simulation. All results are purely simulated. This report validates the solver against the forward model, not the model against reality. Next steps should include validating the Hybrid solver against physical data.

Neural Network Architecture. A single, basic neural network architecture was used. There was no capacity study. Future work could include testing more complex models, seeing how much training data is needed, and varying model capacity to tune for accuracy and computation time in inference.

Jacobian Calculation. This report uses a finite-difference Jacobian. However, if the forward map were reimplemented in PyTorch, we could have used autograd to obtain an analytic Jacobian.

Bibliography

[1] Sezgin Secil and Metin Ozkan. A collision-free path planning method for industrial robot manipulators considering safe human–robot interaction. *Intelligent Service Robotics*, 16:323–359, 2023.

[2] Herman Alexander Jaeger, Alfred Michael Franz, Kilian O'Donoghue, Alexander Seitel, Fabian Trauzettel, Lena Maier-Hein, and Pádraig Cantillon-Murphy. Anser EMT: the first open-source electromagnetic tracking platform for image-guided interventions. *International Journal of Computer Assisted Radiology and Surgery*, 12:1059–1067, 2017.

[3] David J. Griffiths. *Introduction to Electrodynamics*. Pearson, Boston, 4 edition, 2013.

[4] Benjamin McKay. *Lectures on Differential Geometry*. University College Cork, Cork, Ireland, 2022. Lecture notes, University College Cork.

[5] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer Series in Operations Research and Financial Engineering. Springer, New York, 2 edition, 2006.

[6] Kenneth Levenberg. A method for the solution of certain non-linear problems in least squares. *Quarterly of Applied Mathematics*, 2:164–168, 1944.

[7] Donald W. Marquardt. An algorithm for least-squares estimation of non-linear parameters. *Journal of the Society for Industrial and Applied Mathematics*, 11(2):431–441, 1963.

[8] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017.

[9] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts, 2017.

.1 Mapping between python modules and report sections